```
        score.append(ep_len)
        reward_batch = torch.Tensor([r for (s,a,r) in
        transitions]).flip(dims=(0,))
10
        disc_rewards = discount_rewards(reward_batch)
11
        state_batch = torch.Tensor([s for (s,a,r) in transitions])
12
        action_batch = torch.Tensor([a for (s,a,r) in transitions])
13
        pred_batch = model(state_batch)
14
        prob_batch = pred_batch.gather(dim=1,index=action_
        batch.long().view(-1,1)).squeeze()
15
        loss = loss_fn(prob_batch, disc_rewards)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

- ***1*** A list to keep track of the episode length over training time
- ***2*** A list of state, action, rewards (but we ignore the reward)
- ***3*** While in the episode
- ***4*** Gets the action probabilities
- ***5*** Selects an action stochastically
- ***6*** Takes the action in the environment
- ***7*** Stores this transition
- ***8*** If the game is lost, breaks out of the loop
- ***9*** Stores the episode length
- ***10*** Collects all the rewards in the episode in a single tensor
- ***11*** Computes the discounted version of the rewards
- ***12*** Collects the states in the episode in a single tensor
- ***13*** Collects the actions in the episode in a single tensor
- ***14*** Recomputes the action probabilities for all the states in the episode
- ***15*** Subsets the action-probabilities associated with the actions that were actually taken

We start an episode, use the policy network to take actions, and record the states and actions we observe. Then, once we break out of an episode, we have to recompute the predicted probabilities to use in our loss function. Since we record all the transitions in each episode as a list of tuples, once we’re out of the episode we can separate each component of each transition